# ANIL NEERUKONDA INSTITUTE OF TECHNOLOGY & SCIENCES

## (AUTONOMOUS)

*(Permanent Affiliation by Andhra University & Approved by AICTE)*

*Accredited by NBA (ECE, EEE, CSE, IT, Mech, Civil & Chemical) & NAAC with A+ Grade) Sangivalasa-531162,*

*Bheemunipatnam Mandal, Visakhapatnam District*

## A MINI PROJECT WORK ON..

## CAMPUS ROUTE NAVIGATION USING GRAPH TRAVERSALS

*This project is submitted by*

**Team Number: 4**


| | |
|---|---|
| A25126510012 | G. SWATHI |
| A25126510017 | GDS. HARSHINI |
| A25126510040 | POOJA SINGH |
| A25126510041 | P. SRI VARSHINI |
| A25126510066 | Y. UJWALA |


**Under the Guidance of Dr.B.Meena**

Department of Computer Science and Engineering *in*

*partial fulfilment of the requirements for the award of*

**ANIL NEERUKONDA INSTITUTE OF TECHNOLOGY AND SCIENCES**

Bachelor of Technology in

**COMPUTER SCIENCE AND ENGINEERING**

**2025-2029**

**Dr.B.Meena**
**Faculty Incharge**

**Prof.G.Srinivas**
**Head of the Department**
**Computer Science and Engineering**

# CERTIFICATE

This is to certify that the project document entitled “CAMPUS ROUTE NAVIGATION USING GRAPH TRAVERSALS” submitted by the following students of II/IV B.Tech – CSE, Academic Year 2026–2027, is a bonafide record of the work carried out by them under my supervision and guidance.

A25126510012 G.Swathi
A25126510017 G.D.S.Harshini
A25126510040 Pooja Singh
A25126510041 P.Sri Varshini
A25126510066 Y.Ujwala

The document has been submitted in partial fulfillment of the requirements for the course Data Structures and Algorithms, and has not been submitted to any other university or institution for the award of any degree or diploma.

ANITS, Visakhapatnam

Signature
**Head of the Department** Department of

***Dr. B .Meena***

CSE ANITS, Visakhapatnam Signature

Faculty, Data Structures and Algorithms

Date: Place: Visakhapatnam

# DECLARATION

We hereby declare that the project work entitled “CAMPUS ROUTE NAVIGATION USING GRAPH TRAVERSALS” is an original piece of work carried out by us under the guidance of ***Dr. B. Meena***, Faculty of Data Structures and Algorithms, Anil Neerukonda Institute of Technology and Sciences (ANITS), Visakhapatnam.

We further declare that:

· The work reported in this document is entirely our own and has not been copied from any other source without due acknowledgement.

· All results and behaviour described were obtained by running the implemented programs, and the algorithms follow standard descriptions from the literature.

· This document has not been submitted to any other university or institution, either in full or in part, for the award of any degree, diploma, or certificate.

Place: Visakhapatnam Date:

A25126510012 G.Swathi
A25126510017 G.D.S.Harshini
A25126510040 Pooja Singh
A25126510041 PSri Varshini
A25126510066 Y.Ujwala

# ACKNOWLEDGEMENT

The successful completion of this project would not have been possible without the support and guidance of several people, to whom we owe our sincere gratitude.

We express our deepest gratitude to Dr. _____, Faculty of Data Structures and Algorithms, ANITS, for her guidance, patience and constructive feedback at every stage of this project.

We extend our sincere thanks to the Head of the Department of Computer Science and Engineering, ANITS, for providing the academic environment, laboratory facilities and institutional support needed to carry out this project.

We are also grateful to the Management of ANITS for encouraging a culture of learning and innovation among students, and for the computing facilities that made our development and testing possible.

Finally, we thank our family members and friends for their continuous support, encouragement and patience throughout the project.

# ABSTRACT

As a college campus grows, finding the shortest and most convenient route between different buildings can become difficult, especially for new students and visitors. Manually identifying routes may take time, and it can be challenging to understand how different campus locations are connected. This project, the **Campus Route Navigation System**, is a console-based application written in C that addresses these problems using a **Graph data structure and graph traversal techniques**.

In this system, each important campus location is represented as a vertex, while the paths connecting two locations are represented as edges. The campus locations may include CSE, Mechanical, IT, EEE, ECE, CSM, CSD, canteen, and the bus waiting area. The system uses **Depth First Search (DFS)** and **Breadth First Search (BFS)** to explore the campus graph and determine whether a route exists between two selected locations. BFS explores the graph level by level, while DFS explores one path as deeply as possible before backtracking.

The system provides operations such as adding and displaying campus locations and paths, performing BFS and DFS traversals, and checking connectivity between selected locations. The project demonstrates the practical use of **structures, arrays, functions, pointers, and graph traversal techniques in C**. It provides a simple way to represent campus connectivity and helps users understand how graph algorithms can be applied to real-world navigation problems.

The report explains the problem and objectives, existing and proposed approaches, system architecture, graph representation, BFS and DFS modules, implementation, sample outputs, testing, complexity analysis, advantages, limitations, future scope, and conclusion. The main contribution of the project is the application of standard graph traversal algorithms to a practical campus navigation problem, providing a simple and understandable solution for exploring routes between different campus locations.



**Keywords:** Graph, Campus Navigation, Breadth First Search, Depth First Search, Graph Traversal, Vertices, Edges, Connectivity, C Programming.


.

# Campus Route Navigation Using Graph Traversal

## 1. Aim:

The aim of this project is to develop a **Campus Route Navigation System using Graph Traversal** in the C programming language. The system represents the campus locations of ANITS as vertices or nodes in a graph. The roads and paths connecting these locations are represented as edges. Graph traversal algorithms, Breadth First Search (BFS) and Depth First Search (DFS), are used to explore the campus. BFS helps users find a suitable route between two different locations by using the minimum number of roads, and DFS explores all the locations that can be reached from a selected starting location. It demonstrates the practical application of graph data structures in real-life navigation. It makes finding campus routes easier, faster, and more organized. Overall, the project provides an efficient approach for navigating through a campus.

## 2. Problem Statement:

Navigating through a large college campus can be difficult for students, faculty members, visitors, and especially new students who are unfamiliar with the locations of different buildings and facilities. The ANITS campus modelled in this project contains thirteen locations: Main Gate, Open Auditorium, Canteen, Admin Block, CSE, ECE, EEE, Chemical, Mechanical, Food Engine, Bus Area, Hostels and Back Gate, with multiple roads connecting them. Finding a suitable route manually can be time-consuming and confusing.

To solve this problem, the campus can be represented using a Graph Data Structure, where each campus location is considered a vertex and each road connecting two locations is considered an edge. Since the roads in the proposed system allow movement in both directions, an undirected graph is used. An adjacency list is used to efficiently store and manage the connections between different campus locations. In the C program, the adjacency list of each location is stored in an integer array inside a structure.

The proposed system uses Breadth First Search (BFS) to find the shortest route between two campus locations based on the number of roads, while Depth First Search (DFS) is used to explore the connected locations from a selected starting point. The system provides a simple menu-driven interface to display locations, view the campus map, find routes, and explore the campus, thereby demonstrating the practical application of graph traversal techniques in Data Structures and Algorithms.

## 3. Introduction

### 3.1 Campus Navigation

A college campus consists of many important locations such as the main gate, academic blocks, laboratories, library, canteen, hostels, auditorium, and playgrounds. Students, faculty members, and visitors may need to travel between these locations frequently. A campus navigation system helps users identify a route from one location to another in a simple and systematic manner.

### 3.2 Need for Campus Route Navigation

Finding a route inside a large campus can be difficult, particularly for new students and visitors. A navigation system can reduce the time required to locate different buildings and facilities. It can provide a systematic method for finding a path between a given source location and destination location.

### 3.3 Graph-Based Navigation

A graph is a data structure consisting of vertices and edges. In the campus navigation system, each important campus location is represented as a vertex, while a road or pathway connecting two locations is represented as an edge.

For example, the Main Gate, Admin Block, CSE, Canteen, and Hostels can be represented as vertices. If there is a direct path between two locations, an edge is created between their corresponding vertices. In this way, the entire campus can be represented as a graph.

### 3.4 Graph Traversal

Graph traversal is the process of visiting the vertices of a graph systematically. It is an important technique used for exploring connected locations and finding routes.

The two commonly used graph traversal techniques are:

- **Breadth First Search (BFS)**
- **Depth First Search (DFS)**

BFS explores nearby locations first before moving to locations farther away. DFS explores one path deeply before returning and exploring another path.

### 3.5 Role of BFS in Navigation

Breadth First Search is useful for finding a route between two locations in an unweighted graph. It visits the neighboring locations first and then continues to the next level.

For campus navigation, BFS can be used to find a route with the minimum number of edges or road segments between a source and destination when all paths are considered equal. In this project, BFS uses an array-based queue and a parent array, so that the route can be rebuilt and displayed once the destination is reached.

### 3.6 Role of DFS in Navigation

Depth First Search explores a graph by following one path as deeply as possible before backtracking. DFS can be used to explore the connectivity of different campus locations.

In this project, DFS is implemented recursively. Starting from a location chosen by the user, it visits every campus location that can be reached and prints the locations in the order in which they are visited. The DFS in this program explores the campus; it does not search for a particular destination.

### 3.7 Objective of the Project

The main objective of this project is to develop a simple campus route navigation system using graph traversal techniques. The project represents campus locations as vertices and connecting paths as edges, and applies BFS and DFS to explore and find routes.

The specific objectives are:

- To represent the predefined ANITS campus locations and roads as an undirected graph using array-based adjacency lists.
- To find a route with the minimum number of roads between two locations using BFS.
- To explore all reachable campus locations from a chosen starting location using recursive DFS.
- To provide a menu-driven interface for displaying locations, displaying the campus map, finding routes and exploring the campus.
- To understand the practical use of structures, arrays, queues and recursion in a graphbased application.

The project provides a practical understanding of graph data structures and demonstrates their application in solving real-world route navigation problems.

# 4. Graph

## 4.1 Definition of Graph

A graph is a non-linear data structure that consists of a set of **vertices (nodes)** and **edges (connections)**. Graphs are commonly used to represent relationships and connections between different objects.

In the Campus Route Navigation system, the graph is used to represent different locations and the paths connecting them. Each campus location is represented as a vertex, and each road or pathway is represented as an edge.

## 4.2 Vertices

A vertex, also called a node, represents an individual location or object in a graph.

In the proposed campus navigation system, vertices represent the campus locations stored in the program:

- Main Gate
- Open Auditorium
- Canteen
- Admin Block
- CSE, ECE, EEE, Chemical and Mechanical (department blocks)
- Food Engine
- Bus Area
- Hostels
- Back Gate

In the program, each vertex is identified by its position (index) in the array `graph[]`. For example, Main Gate has index 0, Open Auditorium has index 1, Admin Block has index 3 and Back Gate has index 12.

## 4.3 Edges

An edge represents a connection between two vertices. In a campus navigation graph, an edge represents a road, pathway, or direct connection between two locations.

For example, there is a direct road between the Main Gate and the Admin Block, so an edge is created between their corresponding vertices. The Admin Block is in turn connected to CSE. The graph can be represented as:

**Main Gate — Admin Block — CSE**

Here, the connections between the locations represent the edges of the graph. Because every road in this project can be travelled in both directions, each edge is stored for both of its end vertices.

## 4.4 Directed and Undirected Graphs

Graphs can be classified as directed and undirected graphs.

### Directed Graph

In a directed graph, edges have a specific direction. A path from location A to location B does not necessarily mean that a path exists from B to A.

For example:

$$A \rightarrow B$$

This indicates movement from A to B only.

**Undirected Graph**

In an undirected graph, the connection works in both directions. If A is connected to B, then B is also connected to A.

For campus navigation, an undirected graph is used when roads or pathways allow movement in both directions. **This project uses an undirected graph:** the function addRoad() stores the road in the adjacency array of both locations, so that if Main Gate is connected to Admin Block, then Admin Block is also connected to Main Gate.

### 4.5 Weighted and Unweighted Graphs

Graphs can also be classified as weighted and unweighted graphs.

**Weighted Graph**

In a weighted graph, each edge has a value called a weight. The weight may represent distance, travelling time, or cost.

For example (illustration only, not part of this project):

CSE — 200 m — Admin Block

Here, 200 m would represent the distance between the two locations.

**Unweighted Graph**

In an unweighted graph, all edges are treated equally and do not have specific weights.

This project uses an unweighted graph. The program stores no distance or travelling time for any road, so BFS compares routes only by the number of roads they contain.

### 4.6 Graph Representation

A graph can be represented using different data structures. The two commonly used methods are:

1. Adjacency Matrix
2. Adjacency List

**Adjacency Matrix**

An adjacency matrix represents a graph using a two-dimensional array. Each row and column represents a vertex. A value indicates whether two vertices are connected.

For example, if A and B are connected, the corresponding position in the matrix contains 1. If they are not connected, it contains 0.

**Adjacency List**

An adjacency list stores the neighboring vertices of each vertex. It is useful when a graph has many vertices but relatively fewer connections.

For example, part of the campus graph of this project is stored as:

```
Main Gate       -> Open Auditorium, Canteen, Admin Block
Open Auditorium -> Main Gate
Canteen         -> Main Gate
Admin Block     -> Main Gate, CSE, ECE, EEE, Chemical, Mechanical
```

> This representation makes it easy for BFS and DFS to identify the neighboring campus locations.

### 4.7 Graph in Campus Route Navigation

In this project, the campus is represented as a graph where:

- Vertices represent campus locations.
- Edges represent roads or pathways.

- Source vertex represents the starting location.
- Destination vertex represents the location the user wants to reach.
- Traversal algorithm searches the graph to find a route.

For example, if the user wants to travel from the Main Gate to Food Engine, BFS starts from the Main Gate and traverses the connected locations level by level until it reaches Food Engine.

Thus, graph data structures provide the basic foundation for implementing the Campus Route Navigation system.

## 5. CAMPUS REPRESENTATION USING GRAPH

A campus contains many different locations such as the main gate, academic blocks, canteen, hostels, parking or bus areas, and administrative offices. Finding a route between these locations can be represented using a graph.

In this project, the campus is converted into a graph structure. The important locations in the campus are represented as vertices (nodes), while the roads or paths connecting those locations are represented as edges. Graph traversal algorithms such as Breadth First Search (BFS) and Depth First Search (DFS) can then be used to navigate through the campus.

### 5.1 Campus Locations as Vertices

In a graph, a vertex represents a particular location or point. In campus route navigation, each important campus location is considered as a vertex. The program defines the following 13 locations, in the order in which they are added by addLocation(). Each location is identified by a unique name and by its index in the array graph[], so that it can be easily represented in the program.

| Index | Location | Index | Location |
|---|---|---|---|
| 0 | Main Gate | 7 | Chemical |
| 1 | Open Auditorium | 8 | Mechanical |
| 2 | Canteen | 9 | Food Engine |
| 3 | Admin Block | 10 | Bus Area |
| 4 | CSE | 11 | Hostels |
| 5 | ECE | 12 | Back Gate |
| 6 | EEE | | |

Menu option 1 numbers the locations from 1 to 13 (index + 1).

### 5.2 Roads/Paths as Edges

An edge represents a connection between two vertices. In campus navigation, an edge represents a road, path, corridor, or walkway connecting two campus locations.

For example, the Main Gate is connected to the Admin Block, so an edge is created between these two vertices. The Admin Block is also connected to CSE:

```
Main Gate -------- Admin Block -------- CSE
  (0)              (3)              (4)
```

The edges show which locations can be reached directly from another location. The program defines 33 addRoad() calls in main(). Ten of these repeat a road that has already been added (for example, the road between ECE and CSE is requested again after CSE–ECE), and addRoad() ignores such duplicates. The campus graph therefore contains 13 vertices and 23 edges (46 adjacency entries in total).

### 5.3 Adjacency Matrix

An adjacency matrix is a two-dimensional array used to represent a graph. Each row and column represents a vertex. If two locations are directly connected, the corresponding matrix position contains 1. If they are not connected, it contains 0.

For example, consider the following small set of connections between four locations A, B, C and D (A–B, A–C, B–D, C–D). The adjacency matrix can be represented as:

| | **A** | **B** | **C** | **D** |
|---|---|---|---|---|
| A | 0 | 1 | 1 | 0 |
| B | 1 | 0 | 0 | 1 |
| C | 1 | 0 | 0 | 1 |
| D | 0 | 1 | 1 | 0 |

- 1 means a connection exists.
- 0 means there is no direct connection.

The adjacency matrix makes it easy for a program to check whether two campus locations are directly connected. For the 13 locations of this campus, a matrix would need 13 × 13 = 169 entries, although the graph has only 23 roads.

### 5.4 Adjacency List

An adjacency list is another method of representing a graph. In this method, each vertex stores a list of the vertices directly connected to it.

For the same small example, the adjacency list can be represented as:

```
A -> B, C
B -> A, D
C -> A, D
D -> B, C
```

An adjacency list is useful when the campus has many locations but only a limited number of connections between them. **This is the representation used in the program.** The adjacency list of the actual campus graph, in the order in which neighbours are stored (and printed by menu option 2), is:

| **Location** | **Adjacent locations (stored order)** |
|---|---|
| Main Gate | Open Auditorium, Canteen, Admin Block |
| Open Auditorium | Main Gate |
| Canteen | Main Gate |
| Admin Block | Main Gate, CSE, ECE, EEE, Chemical, Mechanical |
| CSE | Admin Block, ECE, EEE, Chemical, Mechanical |
| ECE | Admin Block, CSE, EEE, Chemical, Mechanical |

| | |
|---|---|
| EEE | Admin Block, CSE, ECE, Chemical, Mechanical |
| Chemical | Admin Block, CSE, ECE, EEE, Mechanical |
| Mechanical | Admin Block, CSE, ECE, EEE, Chemical, Food Engine, Bus Area |
| Food Engine | Mechanical, Bus Area |
| Bus Area | Mechanical, Food Engine, Hostels, Back Gate |
| Hostels | Bus Area |
| Back Gate | Bus Area |

### 5.5 Example of Campus Graph

The complete campus graph defined in the program is shown in Figure 5.1. Each box shows a location and its array index. The five department blocks CSE, ECE, EEE, Chemical and Mechanical are connected to each other, and each of them is connected to the Admin Block. The Main Gate, Open Auditorium and Canteen form the entrance side of the campus, while Mechanical links the academic side with the Food Engine, Bus Area, Hostels and Back Gate.

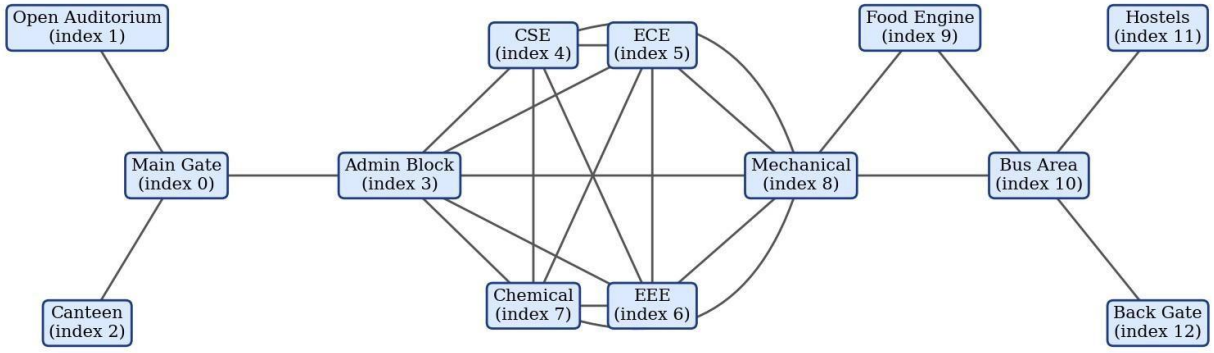



*Figure 5.1: Campus graph defined in the program (13 vertices, 23 edges)*

### 5.6 Graph Traversal in Campus Navigation

Once the campus is represented as a graph, graph traversal algorithms can be applied to visit the locations.

The two main traversal techniques used in this project are:

**Breadth First Search (BFS)**

BFS visits the graph level by level. It starts from a selected location and visits its nearby connected locations first.

For example, if the user wants to travel from the Main Gate to Food Engine, BFS explores the connected locations and finds the route Main Gate → Admin Block → Mechanical → Food Engine.

**Depth First Search (DFS)**

DFS explores one path as deeply as possible before going back and exploring another path.

In this project, DFS is used to traverse the campus graph and list all the locations that can be reached from the selected starting location.

### 5.7 Importance of Graph Representation

Representing the campus as a graph provides a simple and systematic way to model campus routes. It helps the navigation system understand the relationships between different locations. The graph representation is used to:

- Store campus locations.
- Store connections between locations.
- Find routes between two locations.
- Check whether a destination is reachable (BFS reports when no route exists).
- Traverse different campus locations.
- Apply BFS and DFS algorithms.
- Display the navigation path to the user.

Thus, converting the campus into a graph is an important step in developing the Campus Route Navigation Using Graph Traversal system.

## 6. ALGORITHMS USED

Algorithms are a set of step-by-step instructions used to solve a particular problem. In the Campus Route Navigation Using Graph Traversal project, graph traversal algorithms are used to visit campus locations and find a route between different locations.

The main algorithms used in this project are:

1. Breadth First Search (BFS)
2. Depth First Search (DFS)

### 6.1 Breadth First Search (BFS)

Breadth First Search, commonly called BFS, is a graph traversal algorithm that visits vertices level by level.

BFS starts from a selected source vertex and first visits all the vertices directly connected to it. Then, it visits the next level of connected vertices.

BFS uses a queue data structure. In this program the queue is a simple integer array `queue[MAX]` with the two indices `front` and `rear`.

**Steps of BFS (as implemented in BFS())**

1. Initialise the queue (front = rear = 0), the visited array (all 0) and the parent array (all −1).
2. Insert the starting location into the queue and mark it as visited.
3. While the queue is not empty, remove one location (current) from the front of the queue.
4. If current is the destination, stop the search.
5. Otherwise, for every location connected to current that is not yet visited: mark it as visited, store current as its parent, and insert it into the queue.
6. If the destination was never visited, display that no route exists.
7. Otherwise, follow the parent array from the destination back to the source, and display the route in source-to-destination order together with the number of roads.

**Pseudocode**

```
BFS(start, destination):     for i = 0 to MAX-1: visited[i] = 0, parent[i] = -1
front = 0, rear = 0     queue[rear++] = start; visited[start] = 1     while front
< rear:         current = queue[front++]         if current == destination: break
for each neighbour next of current:             if visited[next] == 0:
visited[next] = 1                parent[next] = current
queue[rear++] = next
   if visited[destination] == 0: print "No route exists"; return
   pathCount = 0; current = destination     while current != -1:         path[pathCount++] =
current; current = parent[current]
   print path from index pathCount-1 down to 0     print pathCount - 1 as the number
of roads
```

**Example of BFS**

If BFS is started from the Main Gate on the campus graph, the locations are discovered level by level as shown below (the distance is the number of roads from the Main Gate):

| Level (roads from Main Gate) | Locations discovered |
|---|---|
| 0 | Main Gate |
| 1 | Open Auditorium, Canteen, Admin Block |
| 2 | CSE, ECE, EEE, Chemical, Mechanical |
| 3 | Food Engine, Bus Area |
| 4 | Hostels, Back Gate |

BFS first visits the locations close to the starting location and then moves to the next level. Note that the program's BFS() function stops as soon as the destination is removed from the queue, and it prints only the route, not the complete discovery order.

**Advantages of BFS**

- Easy to understand and implement.
- Uses a queue.
- Visits nearby locations first.
- Useful for finding a route based on the minimum number of edges in an unweighted graph.
- Suitable for campus graphs where each path is considered equally important.

### 6.2 Depth First Search (DFS)

Depth First Search, commonly called DFS, is another graph traversal algorithm.

DFS starts from a selected vertex and follows one path as deeply as possible. When it reaches a location where there are no unvisited connected locations, it goes back and explores another path.

DFS can be implemented using an explicit stack or using recursion. **This project uses recursion**, so the program's call stack takes the place of an explicit stack.

**Steps of DFS (as implemented in DFSTraversal() and DFS())**

1. Select the starting location and create a visited array with all values 0.

2. Mark the current location as visited and print its name.
3. Go through the neighbours of the current location in their stored order.
4. For each neighbour that is not visited, call DFS() on it (move deeper).
5. When a location has no unvisited neighbour left, the function returns to the previous location (backtracking).
6. Continue until every location reachable from the starting location has been visited.

**Pseudocode**

```
DFSTraversal(start):     visited[0..MAX-1] = 0
DFS(start, visited)

DFS(current, visited):     visited[current] = 1     print name
of current     for each neighbour next of current:          if
visited[next] == 0:             DFS(next, visited)
```

**Example of DFS**

The output of the program for DFS started from Hostels is shown in Section 12. The traversal can be traced as follows. Hostels has only one neighbour, Bus Area. Bus Area leads to Mechanical (its first neighbour), Mechanical leads to Admin Block, Admin Block leads to Main Gate and then to Open Auditorium. Since Open Auditorium has no unvisited neighbour, DFS backtracks to Main Gate and visits Canteen, then backtracks to Admin Block and visits CSE, ECE, EEE and Chemical. DFS then backtracks to Mechanical and visits Food Engine, and finally backtracks to Bus Area and visits Back Gate. The resulting order is:

```
Hostels Bus Area Mechanical Admin Block Main Gate Open Auditorium Canteen CSE ECE EEE   Chemical
Food Engine Back Gate
```

The exact DFS order depends on the order in which adjacent vertices are stored. If DFS is started from Main Gate instead, the order obtained by running the program is: Main Gate, Open Auditorium, Canteen, Admin Block, CSE, ECE, EEE, Chemical, Mechanical, Food Engine, Bus Area, Hostels, Back Gate.

**Advantages of DFS**

- Simple to implement using recursion or a stack.
- Useful for exploring all reachable locations.
- Can be adapted to check whether a destination is reachable.
- Useful for exploring different possible paths in a campus graph.

### 6.3 Route Finding Using BFS

In campus route navigation, BFS is used to find a route between a source and destination. For example, suppose the user wants to travel from:

Source : Admin Block

Destination : Bus Area

The system starts BFS from the Admin Block and explores the connected locations. Mechanical is discovered directly from Admin Block, and Bus Area is then discovered from Mechanical. The route displayed by the program (see Section 12) is:

**Admin Block → Mechanical → Bus Area  (2 roads)**

The system stores the previous location of every visited vertex in the parent array. Once the destination is reached, the stored information is used to reconstruct the route. For the route from Main Gate to Food Engine, parent[Food Engine] = Mechanical, parent[Mechanical] = Admin Block and parent[Admin Block] = Main Gate; reading these back from the destination gives the route Main Gate → Admin Block → Mechanical → Food Engine (3 roads).

Thus, BFS provides a route with the fewest roads between two campus locations because the graph is unweighted. When more than one route has the same number of roads, the route found first, according to the stored adjacency order, is displayed.

### 6.4 Campus Exploration Using DFS

DFS is used in this program for campus exploration. For example, if the source is Hostels, DFS starts at Hostels and follows one available path as deeply as possible. When it cannot go further, it goes back and tries another unvisited neighbour. The process continues until all the locations reachable from the starting location have been visited.

The program's DFS() function does not take a destination and does not stop early or print a route. DFS can in general be adapted to search for a path or to check connectivity, but a path found by DFS is not guaranteed to have the fewest roads, which is why route finding in this project is done by BFS.

### 6.5 Algorithm Selection for the Project

Both BFS and DFS are useful for campus route navigation.

BFS is particularly useful when the campus graph is unweighted and the objective is to find a route with the minimum number of connections.

DFS is useful when the objective is to explore the campus graph and list the locations reachable from a given location.

Therefore, the project uses both BFS and DFS to demonstrate graph traversal and campus route navigation.

### 6.6 Time Complexity

For a graph with V vertices and E edges stored as adjacency lists, both BFS and DFS have a time complexity of:

**O(V + E)**

where:

- V = number of campus locations (vertices)
- E = number of connections or paths (edges)

Each vertex is inserted into the queue (BFS) or entered by a recursive call (DFS) at most once, and each adjacency entry is examined once. Since every undirected road appears in two adjacency arrays, the total number of entries examined is 2E, which is still O(E). For the campus in this project, V = 13 and E = 23 (46 adjacency entries), so a traversal examines only a few dozen entries.

| Operation (function) | Time complexity | Remarks |
|---|---|---|
| findLocation() | O(V) | Linear search; each comparison uses strcmp() on a name of at most 49 characters. |
| addLocation() | O(V) | Calls findLocation() to reject duplicates; then copies the name. |

| | | |
|---|---|---|
| addRoad() | O(V + d) | Two findLocation() calls plus a duplicate check over the d neighbours of the first location. |
| BFS() | O(V + E) | Plus O(MAX) to initialise arrays and O(V) to rebuild the path. |
| DFSTraversal() / DFS() | O(V + E) | Each reachable location is visited once. |
| displayGraph() | O(V + E) | Prints every location and all of its neighbours. |
| displayLocations() | O(V) | Prints every location once. |

### 6.7 Space Complexity and Assumptions

The graph itself needs O(V + E) space (here a fixed array of MAX = 20 structures, each holding a name and an adjacency array of MAX integers). BFS needs the extra arrays queue, visited, parent and path, each of size MAX, which is O(V). DFS needs a visited array and a recursion depth of at most V, which is also O(V).

The following assumptions are made: the graph is unweighted and undirected; the number of locations does not exceed MAX = 20 and each location has at most MAX = 20 neighbours; the cost of one strcmp() call is treated as constant because names are limited to 49 characters; and the O(V + E) bound refers to the traversal itself, not to the user-input and lookup steps.

## 7. WORKING OF THE SYSTEM

The Campus Route Navigation system represents the campus as a graph, where campus locations are represented as vertices and connecting paths are represented as edges. The campus map is predefined in the program: the user does not enter the map, but chooses operations from a menu.

### 7.1 Initialisation

When the program starts, main() calls addLocation() 13 times to create the locations (Main Gate, Open Auditorium, Canteen, Admin Block, CSE, ECE, EEE, Chemical, Mechanical, Food Engine, Bus Area, Hostels and Back Gate). It then calls addRoad() 33 times to connect them. Each road is stored in both directions, and repeated requests for a road that already exists are ignored. This creates the undirected campus graph with 13 vertices and 23 edges.

### 7.2 Menu-Driven Operation

After the graph is created, the program repeatedly displays the menu ANITS CAMPUS ROUTE NAVIGATION inside a while (1) loop and reads the user's choice with scanf(). A switch statement executes the selected operation:

| Choice | Menu option | What the program does | Functions used |
|---|---|---|---|
| 1 | Display Locations | Prints the numbered list of all 13 locations. | displayLocations() |
| 2 | Display Campus Map | Prints each location followed by its adjacent locations. | displayGraph() |
| 3 | Find Shortest Route (BFS) | Shows the locations, reads the starting location and destination, and prints the route with the fewest roads and the number of roads. | displayLocations(), findLocation(), BFS() |

| | | | |
|---|---|---|---|
| 4 | Explore Campus (DFS) | Shows the locations, reads a starting location, and prints all reachable locations in DFS order. | displayLocations(), findLocation(), DFSTraversal(), DFS() |
| 5 | Exit | Prints a thank-you message and terminates the program. | exit(0) |
| other | (invalid choice) | Prints "Invalid choice! Please try again." and shows the menu again. | default case of switch |

### 7.3 Finding a Route (Option 3)

The user types the starting location and the destination as full names (names may contain spaces, for example “Admin Block”). findLocation() converts each name into an index. If either name is not found, the program prints “Invalid location name!”. Otherwise BFS() searches for the destination, rebuilds the route using the parent array, and prints the route and the number of roads.

### 7.4 Exploring the Campus (Option 4)

The user types a starting location. If the name is valid, DFSTraversal() creates a visited array and calls the recursive DFS() function, which prints the name of every location reachable from the start in the order in which it is visited.

**Main Steps**

1. **Create Graph** – Add the campus locations and roads as vertices and edges.
2. **Display Menu** – Show the five menu options and read the user’s choice.
3. **Enter Locations** – For route finding, enter the source and destination; for exploration, enter the starting location.
4. **Apply Algorithm** – Use BFS (option 3) or DFS (option 4) to traverse the graph.
5. **Find Route / Explore** – BFS searches for the destination; DFS visits all reachable locations.
6. **Display Result** – Show the route and number of roads, or the DFS visiting order, and return to the menu.

## 8. MODULES INVOLVED

The program is organised into the following modules. Each module is implemented by one or more C functions. The table gives an overview; the sections that follow describe each module in terms of its purpose, working, relevant functions and contribution to the system.

| No. | Module | Function(s) |
|---|---|---|
| 1 | Location Management Module | addLocation() |
| 2 | Location Search Module | findLocation() |
| 3 | Graph Construction Module | addRoad() |
| 4 | Campus Map Display Module | displayGraph() |
| 5 | Location Listing Module | displayLocations() |

| | | |
|---|---|---|
| 6 | Shortest Route Finding Module (BFS) | BFS() |
| 7 | Campus Exploration Module (DFS) | DFSTraversal(), DFS() |
| 8 | Menu-Driven User Interface Module | main() |
| 9 | Input Validation and Error Handling Module | checks inside addLocation(), addRoad(), BFS() and main() |

### 8.1 Location Management Module

**Purpose:** To add campus locations (vertices) to the graph.

**How it works:** addLocation() first calls findLocation(); if the name already exists it prints "Location already exists!" and returns. It then checks whether locationCount has reached MAX (20); if so it prints "Maximum locations reached!" and returns. Otherwise it copies the name into graph[locationCount].name with strcpy(), sets adjCount of the new location to 0 and increments locationCount. main() calls it 13 times to create the ANITS campus locations.

**Relevant function(s):** addLocation(); uses findLocation(), strcpy(), graph[] and locationCount.

**Contribution to the system:** It creates the vertices of the graph, and prevents duplicate vertices and overflow of the fixed-size array.

### 8.2 Location Search Module

**Purpose:** To find the array index of a location from its name.

**How it works:** findLocation() performs a linear search over the first locationCount entries of graph[], comparing each stored name with the given name using strcmp(). It returns the index of the first match, or −1 if no location has that name. The comparison is case-sensitive and requires the exact name.

**Relevant function(s):** findLocation().

**Contribution to the system:** Every other module works with indices, so this module connects the names typed by the user (and used in main()) with the indices used in the graph. It also supports validation, since a result of −1 means that the name is not a valid location.

### 8.3 Graph Construction Module

**Purpose:** To connect two locations with a road, i.e. to add an edge to the graph.

**How it works:** addRoad() finds the indices u and v of the two names. It rejects the request if either location does not exist or if u equals v, then checks whether v is already in the adjacency array of u (an existing road is silently ignored). It also checks that neither adjacency array is full. Finally it stores v in graph[u].adj, increments graph[u].adjCount, stores u in graph[v].adj and increments graph[v].adjCount. Storing the road in both directions makes the graph undirected.

**Relevant function(s):** addRoad(); uses findLocation().

**Contribution to the system:** It builds the adjacency lists that BFS, DFS and the map display read. main() uses it to define all roads of the campus.

### 8.4 Campus Map Display Module

**Purpose:** To display the campus graph: every location with the locations directly connected to it.

**How it works:** displayGraph() prints the heading “ANITS CAMPUS MAP” and, for each location i, prints its name followed by “->” and the names of all its neighbours, obtained through graph[graph[i].adj[j]].name and separated by commas.

**Relevant function(s):** displayGraph(); menu option 2.

**Contribution to the system:** It lets the user see the adjacency list of the whole campus and check which locations are directly connected before choosing a route.

### 8.5 Location Listing Module

**Purpose:** To display all available campus locations so that the user knows which names can be entered.

**How it works:** displayLocations() prints the heading “ANITS LOCATIONS” and a numbered list (index + 1) of the names of all locations stored in graph[].

**Relevant function(s):** displayLocations(); menu option 1, and automatically before the inputs of options 3 and 4.

**Contribution to the system:** It guides the user while entering location names and reduces invalid inputs.

### 8.6 Shortest Route Finding Module (BFS)

**Purpose:** To find a route between two locations that uses the minimum number of roads.

**How it works:** BFS() uses an array-based queue, a visited array and a parent array (all of size MAX). The start location is inserted into the queue and marked visited. Locations are removed from the front of the queue; each unvisited neighbour is marked visited, its parent is recorded, and it is inserted at the rear. The loop ends when the destination is removed or the queue becomes empty. If the destination was not visited, “No route exists” is printed. Otherwise the route is rebuilt by following the parent array from the destination back to the start into the path array, which is then printed in reverse order (source to destination) with the number of roads, pathCount - 1.

**Relevant function(s):** BFS(); menu option 3.

**Contribution to the system:** It is the route-finding feature of the system. Because the graph is unweighted, the route has the minimum number of edges; the program does not calculate physical distance or travelling time.

### 8.7 Campus Exploration Module (DFS)

**Purpose:** To explore the campus by visiting every location reachable from a chosen starting location.

**How it works:** DFSTraversal() creates a visited array initialised to 0, prints the heading “DFS CAMPUS EXPLORATION” and calls DFS() on the start location. DFS() marks the current location as visited, prints its name, and recursively calls itself for each unvisited neighbour in stored order. When a location has no unvisited neighbours, the function returns and the traversal backtracks.

**Relevant function(s):** DFSTraversal() and DFS(); menu option 4.

**Contribution to the system:** It demonstrates depth-first traversal and shows which locations are reachable and in what visiting order.

### 8.8 Menu-Driven User Interface Module

**Purpose:** To interact with the user and call the required operation.

**How it works:** After creating the graph, main() runs an infinite while (1) loop that prints the menu and reads the choice with scanf(“%d”). A switch statement calls displayLocations() (1), displayGraph()

(2), the route-finding code (3), the DFS code (4) or exits (5). For options 3 and 4 the names are read with scanf(“ %[^\n]”), which accepts names containing spaces.

**Relevant function(s):** main().

**Contribution to the system:** It ties all the modules together and provides the only interface to the user (a text-based console interface).

### 8.9 Input Validation and Error Handling Module

**Purpose:** To prevent invalid data from damaging the graph or causing wrong results, and to give the user clear messages.

**How it works:** The checks are spread over addLocation(), addRoad(), BFS() and main(); they are listed in the table below. **Contribution to the system:** it keeps the graph consistent and stops invalid names or menu choices from reaching the traversal functions.

| Situation | Where handled | Behaviour |
|---|---|---|
| Duplicate location | addLocation() | Prints "Location already exists!" and does not add it. |
| More than MAX locations | addLocation() | Prints "Maximum locations reached!". |
| Unknown location in a road | addRoad() | Prints "One or both locations not found!". |
| Road from a location to itself | addRoad() | Prints "A location cannot have a road to itself!". |
| Duplicate road | addRoad() | Returns without adding it (no message is printed). |
| Adjacency array full | addRoad() | Prints "Cannot add road: adjacency list is full!". |
| Unknown location name entered by user | main(), options 3 and 4 | Prints "Invalid location name!" and returns to the menu. |
| Destination not reachable | BFS() | Prints "No route exists between ...". (This cannot occur with the predefined map, because the campus graph is connected.) |
| Menu choice other than 1–5 | main(), default case | Prints "Invalid choice! Please try again.". |

The program does not handle non-numeric menu input or location names longer than 49 characters; these points are discussed in Section 11.1.

## 9. DATA STRUCTURES USED IN THE PROJECT

This section describes the data structures that are actually used in the C implementation. The structures discussed earlier in this report only as theory (adjacency matrix, linked-list adjacency, explicit stack, weighted edges) are listed separately in Section 9.11.

| Data structure | Declared / used as | Used by |
|---|---|---|

| | | |
|---|---|---|
| Graph (undirected, unweighted) | graph[] of Location structures | All modules |
| Structure | typedef struct { ... } Location | graph[] |
| Array of structures | Location graph[MAX]; int locationCount | addLocation(), findLocation(), display functions |
| Adjacency list (array-based) | int adj[MAX]; int adjCount (inside Location) | addRoad(), BFS(), DFS(), displayGraph() |
| Queue (array-based) | int queue[MAX]; int front, rear | BFS() |
| Visited array | int visited[MAX] | BFS(), DFSTraversal(), DFS() |
| Parent array | int parent[MAX] | BFS() |
| Path array | int path[MAX]; int pathCount | BFS() |
| Recursion / call stack | recursive calls of DFS() | DFS() |
| Character arrays (strings) | char name[NAME_LEN] | Location, findLocation(), addLocation(), main() |

### 9.1 Graph

**Purpose:** to model the campus. **Vertices** are the campus locations, for example Main Gate, Admin Block and Bus Area. **Edges** are the roads between them, for example Main Gate–Admin Block and Mechanical–Bus Area. The program's graph has 13 vertices and 23 edges.

**Why undirected:** every road can be travelled in both directions, so a road between two locations is one undirected edge. In the program addRoad() stores the road in both adjacency arrays, so the graph is undirected. It is also unweighted, because no distance is stored.

**Representation:** the graph is not a separate object; it is represented by the global array graph[] of Location structures, each of which holds the adjacency list of one vertex (Sections 9.2 to 9.4).

### 9.2 Structure (Location)

```
typedef struct {     char
name[NAME_LEN];     int adj[MAX];
int adjCount; } Location;
```

**Purpose:** to group everything the program needs to know about one campus location (one vertex) in a single unit.

- name[NAME_LEN] – character array (NAME_LEN = 50) holding the location name, for example “Admin Block” (at most 49 characters plus the terminating null character).
- adj[MAX] – integer array (MAX = 20) holding the indices of the locations directly connected to this location.
- adjCount – the number of valid entries currently stored in adj[]. Only adj[0] to adj[adjCount - 1] are meaningful.

For example, Admin Block is stored with name = “Admin Block”, adj = {0, 4, 5, 6, 7, 8, …} and adjCount = 6, which means that it is connected to the locations at indices 0 (Main Gate), 4 (CSE), 5 (ECE), 6 (EEE), 7 (Chemical) and 8 (Mechanical).

### 9.3 Array of Structures

**Declaration:** Location graph[MAX]; and int locationCount = 0;

**Purpose and use:** the array graph[] stores all campus locations; the position of a location in the array is its vertex number (index). Because it is a global array, it is available to every function. The variable locationCount records how many entries are in use: addLocation() stores a new location at graph[locationCount] and then increments the counter, so valid indices run from 0 to locationCount - 1. All loops that go over the locations (for example in findLocation(), displayGraph() and displayLocations()) stop at locationCount, and addLocation() refuses to add a location when locationCount has reached MAX. The campus uses 13 of the 20 available entries.

### 9.4 Adjacency List

**Purpose:** to store the neighbours of every vertex so that BFS and DFS can quickly find the locations directly reachable from the current location.

**How it is used:** the adjacency list of location i is the array graph[i].adj[] with graph[i].adjCount entries. For example, the neighbours of Main Gate (index 0) are stored as adj = {1, 2, 3} (Open Auditorium, Canteen, Admin Block) with adjCount = 3. addRoad() appends entries to the lists, while BFS(), DFS() and displayGraph() read them with loops of the form for (i = 0; i < graph[current].adjCount; i++).

### 9.5 Queue

**Declaration (inside BFS()):** int queue[MAX]; int front = 0; int rear = 0;

**Purpose:** BFS must visit locations in the order in which they were discovered (first in, first out). An array with two indices acts as the queue.

**How it works:** rear is the position where the next location is inserted and front is the position of the next location to be removed. Insertion is queue[rear++] = start (or queue[rear++] = next), and removal is int current = queue[front++]. The queue is empty when front == rear, so the loop condition is while (front < rear). The indices only move forward (the queue is not circular), which is sufficient because each location is inserted at most once (it is marked visited at insertion), so at most MAX locations are ever inserted.

### 9.6 Visited Array

**Declaration:** int visited[MAX] = {0}; (local to BFS() and to DFSTraversal()).

**Purpose and use:** visited[i] is 1 if location i has already been discovered and 0 otherwise. In BFS a location is marked visited when it is inserted into the queue, so it can never be inserted twice. In DFS visited[current] = 1 is set when DFS() is entered, and a neighbour is explored only if !visited[next]. This prevents the traversals from going around cycles forever (for example among the mutually connected CSE, ECE, EEE, Chemical and Mechanical blocks) and keeps the running time at O(V + E). In DFS, the same array is passed to every recursive call, so all calls share it. After BFS, visited[destination] also tells whether a route exists.

### 9.7 Parent Array

**Declaration:** int parent[MAX]; in BFS(), initialised to −1 for all entries.

**Purpose and use:** when BFS discovers a neighbour next from current, it executes parent[next] = current. The array therefore records, for every discovered location, the location from which it was reached. The start location keeps the value −1, which marks the beginning of the route. After BFS, the route is rebuilt by starting at the destination and repeatedly moving to its parent. For example, for Main Gate to Food Engine: parent[Food Engine] = Mechanical, parent[Mechanical] = Admin Block, parent[Admin Block] = Main Gate and parent[Main Gate] = −1.

### 9.8 Path Array

**Declaration:** int path[MAX]; int pathCount = 0; in BFS().

**Purpose and use:** the parent array can only be followed backwards, from the destination to the source, so the locations are stored in path[] in reverse order: path[pathCount++] = current; current = parent[current]; until current becomes −1. For the example above the array holds Food Engine, Mechanical, Admin Block, Main Gate. To show the route in the natural order, the program prints path[] with a loop that runs from pathCount - 1 down to 0, giving Main Gate → Admin Block → Mechanical → Food Engine. The number of roads is pathCount - 1, because a route with n locations has n − 1 roads.

### 9.9 Recursion and the Call Stack

**Purpose:** DFS follows one path as deep as possible and then returns to the previous location to try another path. The recursive function DFS(int current, int visited[]) expresses this directly.

**How it works:** each call to DFS() marks current as visited, prints it and then calls DFS() again for the first unvisited neighbour. Each active call occupies a frame on the program's call stack, holding its own values of current, the loop counter and next. When a location has no unvisited neighbour left, the call returns, its frame is removed from the stack, and execution continues in the previous call at the next neighbour; this return is the backtracking step. The program therefore needs no explicit stack. The recursion depth cannot exceed the number of locations (at most 20).

### 9.10 Character Arrays and String Handling

Location names are stored as null-terminated character arrays: char name[NAME_LEN] in each Location, and the arrays name1[NAME_LEN] and name2[NAME_LEN] in main() for user input. The standard library (<string.h>) functions used are:

- strcmp(graph[i].name, name) in findLocation() – returns 0 when the two strings are equal, so the matching location can be identified.
- strcpy(graph[locationCount].name, name) in addLocation() – copies the given name into the new location.

User input is read with scanf(" %[^\n]", name1), where the leading space skips whitespace and %[^\n] reads a whole line, so names that contain spaces (such as “Open Auditorium”) can be entered. Names must match exactly, including upper and lower case.

### 9.11 Data Structures Discussed Only Theoretically

The following are discussed in the theory sections of this report for comparison, but are **not** used in the program: the **adjacency matrix** (Sections 4.6 and 5.3); **linked-list adjacency lists** (dynamic allocation with pointers); an **explicit stack** for DFS (the program uses recursion instead); and **weighted edges** (the graph is unweighted).

## 10. Flowchart

Figure 10.1 is the general flowchart of the BFS and DFS route-navigation process. In the implemented program, route finding (BFS) takes a source and a destination and rebuilds the route through the parent array. The DFS option takes only a starting location and prints the visiting order of the reachable locations; it has no destination test.

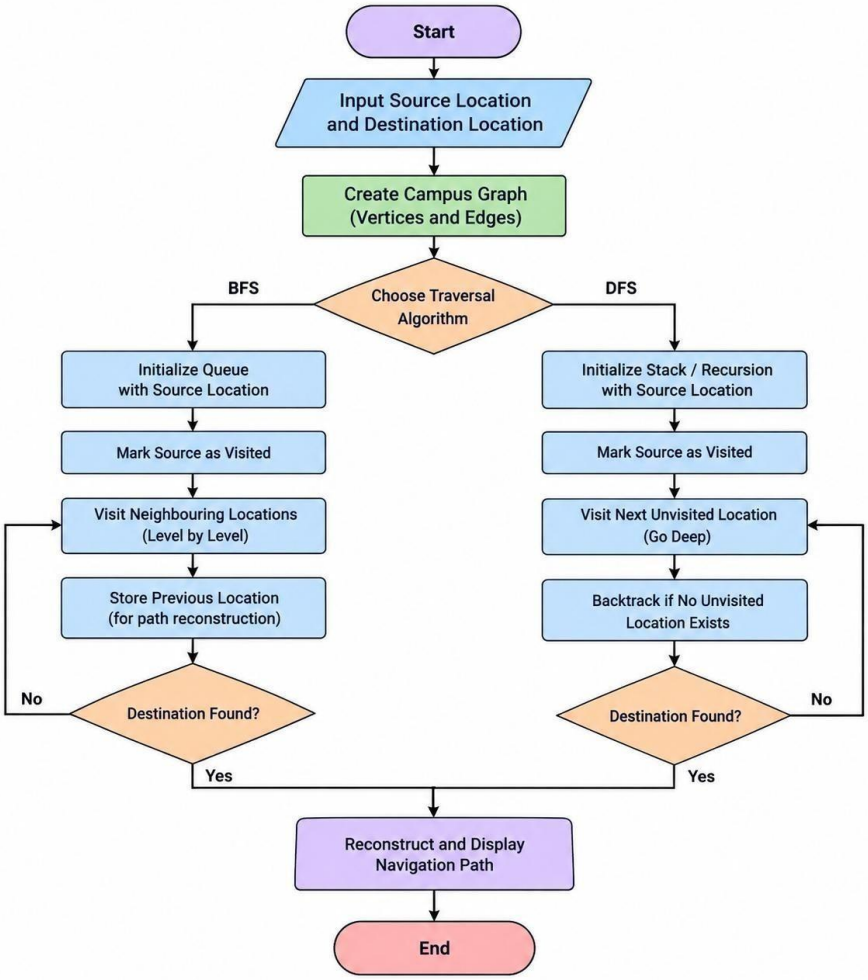



*Figure 10.1: General flowchart of graph-traversal route navigation (original flowchart)*

Figure 10.2 shows the flow of the implemented C program, from the creation of the campus graph to the menu loop and the five options.

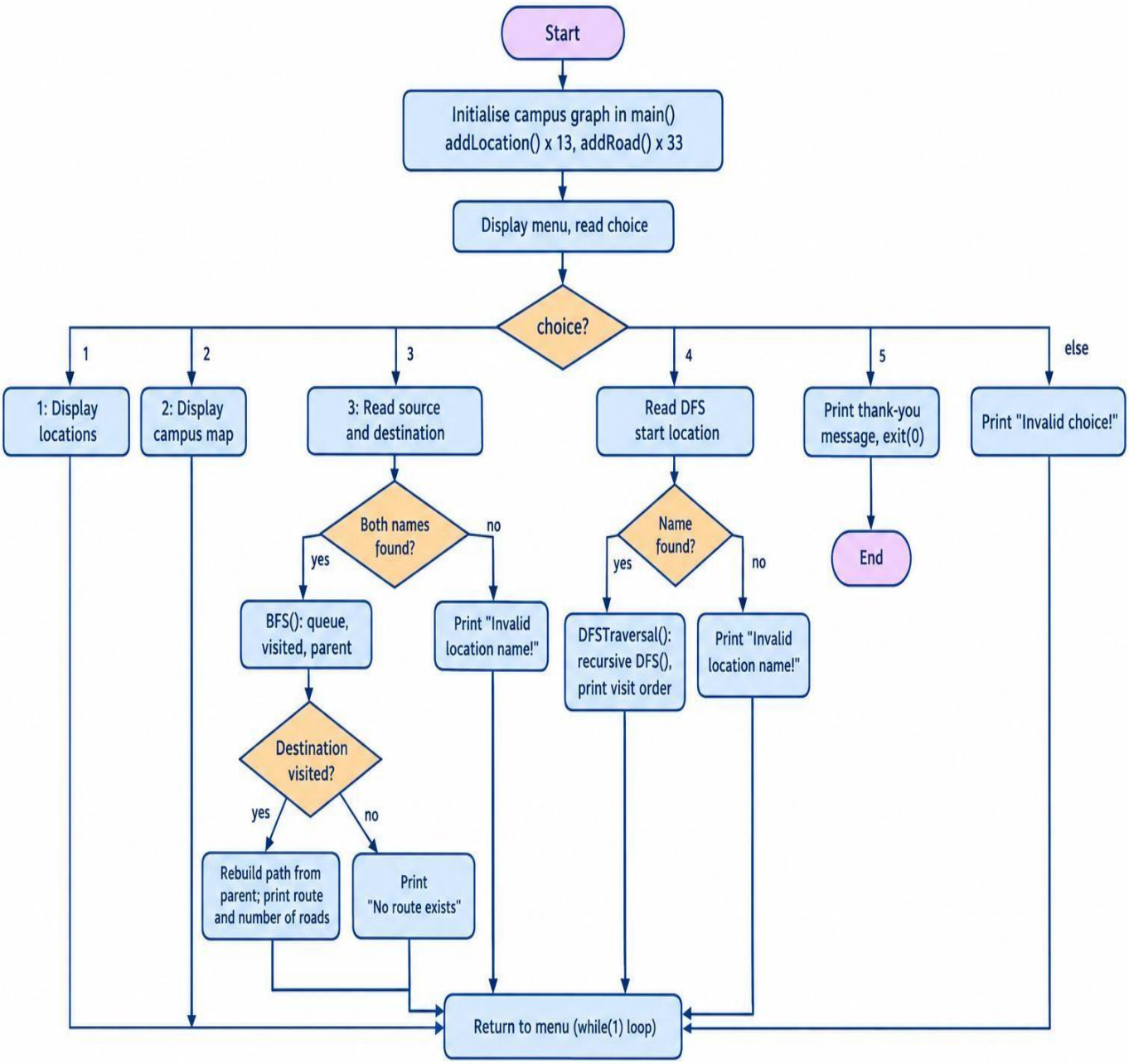



*Figure 10.2: Flowchart of the implemented menu-driven program*

## 11. Source Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX 20
#define NAME_LEN 50
// Graph structure typedef
struct {     char
```

```
name[NAME_LEN];     int
adj[MAX];     int adjCount;
} Location;

Location graph[MAX]; int
locationCount = 0; // Find
location index by name int
findLocation(char name[]) {
   int i;
   for (i = 0; i < locationCount; i++) {         if
(strcmp(graph[i].name, name) == 0) {
          return i;
      }     }
return -1;
}
// Add a new location void
addLocation(char name[]) {     if
(findLocation(name) != -1) {
printf("Location already exists!\n");
return;
   }
   if       (locationCount       >=       MAX)       {
printf("Maximum locations reached!\n");
      return;
   }
   strcpy(graph[locationCount].name, name);
graph[locationCount].adjCount = 0;
locationCount++;
}
// Add a road between two locations void
addRoad(char name1[], char name2[]) {     int u
= findLocation(name1);     int v =
findLocation(name2);     // Check whether both
locations exist     if (u == -1 || v == -1) {
printf("One or both locations not found!\n");
return;
   }
   // Prevent a road from connecting a location to itself
if (u == v) {         printf("A location cannot have a road
to itself!\n");         return;
```

```
    }
    // Check whether the road already exists
for (int i = 0; i < graph[u].adjCount; i++) {
if (graph[u].adj[i] == v) {             return;
        }
    }
    // Check whether both adjacency arrays have space     if
(graph[u].adjCount >= MAX || graph[v].adjCount >= MAX) {
printf("Cannot add road: adjacency list is full!\n");         return;
    }
    // Add the road in both directions (undirected graph)
graph[u].adj[graph[u].adjCount] = v;
graph[u].adjCount++;
graph[v].adj[graph[v].adjCount] = u;
graph[v].adjCount++;
}
// Display complete campus graph void
displayGraph() {
    int i, j;
    printf("\n========== ANITS CAMPUS MAP ==========\n");
    for (i = 0; i < locationCount; i++) {
printf("%s -> ", graph[i].name);         for (j
= 0; j < graph[i].adjCount; j++) {
printf("%s",
graph[graph[i].adj[j]].name);             if (j <
graph[i].adjCount - 1)                printf(", ");
        }
printf("\n");
    }
}
// BFS - finds shortest path
// based on number of roads void
BFS(int start, int destination) {
int queue[MAX];     int front = 0;
int rear = 0;     int visited[MAX] =
{0};     int parent[MAX];
    int i;
    // Initialize parent array
for (i = 0; i < MAX; i++) {
parent[i] = -1;
```

```
    }
    // Add starting location
queue[rear++] = start;     visited[start]
= 1;     // BFS     while (front < rear) {
int current = queue[front++];         //
Destination found         if (current ==
destination)             break;         for (i
= 0;             i <
graph[current].adjCount;             i++)
{             int next =
graph[current].adj[i];             if
(!visited[next]) {
visited[next] = 1;
parent[next] = current;
queue[rear++] = next;
            }
        }
    }
    // Destination not reachable     if
(!visited[destination]) {         printf("\nNo route exists
between %s and %s.\n",                 graph[start].name,
graph[destination].name);         return;
    }
    // Reconstruct path
    int path[MAX];
    int pathCount = 0;     int current
= destination;     while (current !=
-1) {         path[pathCount++] =
current;         current =
parent[current];
    }
    // Display route
    printf("\n========== SHORTEST ROUTE ==========\n");
printf("\n");     for (i = pathCount - 1; i >= 0; i--) {
printf("%s",                 graph[path[i]].name);         if (i != 0)
printf(" -> ");
    }     printf("\n");
printf("\nNumber of roads: %d\n",
pathCount - 1);
    printf("====================================\n");
```

```
}
// DFS function void DFS(int current,
int visited[]) {     int i;
visited[current] = 1;     printf("%s ",
graph[current].name);     for (i = 0;
i < graph[current].adjCount;
i++) {         int next =
graph[current].adj[i];         if
(!visited[next]) {             DFS(next,
visited);
        }
    }
}
// DFS traversal void
DFSTraversal(int start) {
int visited[MAX] = {0};
    printf("\n========== DFS CAMPUS EXPLORATION ==========\n");
    DFS(start, visited);
    printf("\n");
}
// Display all locations void
displayLocations() {
    int i;
    printf("\n========== ANITS LOCATIONS ==========\n");
    for (i = 0; i < locationCount; i++) {
printf("%d. %s\n",
            i           +           1,
graph[i].name);
    }
    printf("=======================================\n");
}
// Main function
int main() {
int choice;
    char  name1[NAME_LEN];          char
name2[NAME_LEN];       // ADD ANITS
CAMPUS                      LOCATIONS
addLocation("Main                   Gate");
addLocation("Open            Auditorium");
addLocation("Canteen");
```

```
addLocation("Admin                    Block");
addLocation("CSE");
addLocation("ECE");
addLocation("EEE");
addLocation("Chemical");
addLocation("Mechanical");
addLocation("Food                     Engine");
addLocation("Bus                        Area");
addLocation("Hostels");
addLocation("Back Gate");
   // ADD ROADS ACCORDING TO YOUR CAMPUS MAP
   // Main Gate connections     addRoad("Main
Gate", "Open Auditorium");     addRoad("Main
Gate", "Canteen");     addRoad("Main Gate",
"Admin Block");
   // Admin Block connections
   addRoad("Admin Block", "CSE");
   addRoad("Admin Block", "ECE");
   addRoad("Admin Block", "EEE");
   addRoad("Admin                Block",                "Chemical");
addRoad("Admin Block", "Mechanical");
   // CSE side     addRoad("CSE",
"ECE");     addRoad("CSE",
"EEE");     addRoad("CSE",
"Chemical");     addRoad("CSE",
"Mechanical");
   // ECE side     addRoad("ECE",
"CSE");     addRoad("ECE",
"EEE");     addRoad("ECE",
"Chemical");     addRoad("ECE",
"Mechanical");
   // EEE side     addRoad("EEE",
"CSE");     addRoad("EEE",
"ECE");     addRoad("EEE",
"Chemical");     addRoad("EEE",
"Mechanical");
   // Chemical side
addRoad("Chemical", "Mechanical");
addRoad("Chemical", "EEE");
```

```
addRoad("Chemical", "ECE");
addRoad("Chemical", "CSE");
   // Mechanical side
addRoad("Mechanical", "Food Engine");
addRoad("Mechanical", "Chemical");
addRoad("Mechanical", "ECE");
addRoad("Mechanical", "EEE");
addRoad("Mechanical", "CSE");
addRoad("Mechanical", "Bus Area");
   // Food Engine side     addRoad("Food
Engine", "Bus Area");
   //       Bus       Area       connections
addRoad("Bus     Area",     "Hostels");
addRoad("Bus Area", "Back Gate");

   while           (1)           {
printf("\n\n");
      printf("================================================\n");
printf("       ANITS CAMPUS ROUTE NAVIGATION\n");
printf("================================================\n");
printf("1. Display Locations\n");        printf("2. Display Campus Map\n");        printf("3.
Find Shortest Route (BFS)\n");        printf("4. Explore Campus (DFS)\n");        printf("5.
Exit\n");
      printf("================================================\n");
printf("Enter your choice: ");        scanf("%d", &choice);        switch (choice) {
         // DISPLAY LOCATIONS
case 1:displayLocations();
break;
         // DISPLAY CAMPUS MAP
case 2:displayGraph();
break;
         // FIND SHORTEST ROUTE
case 3:displayLocations();
printf("\nEnter starting location: ");
scanf(" %[^\n]", name1);
printf("Enter destination: ");
scanf(" %[^\n]", name2);
                  {                    int start =
findLocation(name1);                     int
destination =                    findLocation(name2);
```

```
if (start == -1 ||                    destination == -1) {
printf("\nInvalid location name!\n");
                }
else {
                   BFS(start, destination);
                }
}
break;
        // DFS CAMPUS EXPLORATION
        case 4:displayLocations();
printf("\nEnter starting location for DFS: ");
scanf(" %[^\n]", name1);
            {                int start =
findLocation(name1);                if (start == -1) {
printf("\nInvalid location name!\n");
                }
else {
                   DFSTraversal(start);
                }            }              break;          // EXIT
case 5:printf("\n");                printf("Thank you for
using\n");              printf("ANITS Campus Route
Navigation!\n");                exit(0);
        // INVALID CHOICE
default:printf("\nInvalid choice!\n");
printf("Please try again.\n");
      }    }
return 0;
}
```

### 11.1 Observations on the Source Code

The source code above is the original code and has not been changed in this report. The following points were noticed while analysing it; they do not affect the sample outputs shown in this report, and are listed only so that the limitations of the implementation are stated clearly.

| No. | Observation | Effect |
|---|---|---|
| 1 | The menu choice is read with scanf("%d"). If the user types a nonnumeric value, the input is not consumed and the loop can repeat indefinitely. | Only integer menu input is supported. |
| 2 | Location names are read with scanf(" %[^\n]") without a field width, into arrays of 50 characters. | A name longer than 49 characters could overflow the array. |

| | | |
|---|---|---|
| 3 | Names are compared with strcmp(), so they are case-sensitive and must be typed exactly. | "admin block" is reported as an invalid location name. |
| 4 | addRoad() returns silently when the road already exists. main() requests ten roads twice (for example CSE–ECE and ECE–CSE). | The duplicate requests are harmless; 33 calls create 23 roads. |
| 5 | If the start and destination are the same location, BFS() displays that single location and "Number of roads: 0". | Correct behaviour for a zero-length route. |

| | | |
|---|---|---|
| 6 | addLocation() and addRoad() are called only from main(); the menu has no option to add locations or roads. | The campus map is fixed at compile time. |
| 7 | The comment "ADD ROADS ACCORDING TO YOUR CAMPUS MAP" is a leftover from development. | Cosmetic only. |

## 12. Output:

```
==============================================
       ANITS CAMPUS ROUTE NAVIGATION
==============================================
1. Display Locations
2. Display Campus Map
3. Find Shortest Route (BFS)
4. Explore Campus (DFS)
5. Exit
==============================================
Enter your choice: 1

========== ANITS LOCATIONS ==========
1. Main Gate
2. Open Auditorium
3. Canteen
4. Admin Block
5. CSE
6. ECE
7. EEE
8. Chemical
9. Mechanical
10. Food Engine
11. Bus Area
12. Hostels
13. Back Gate
====================================


==============================================
       ANITS CAMPUS ROUTE NAVIGATION
==============================================
1. Display Locations
2. Display Campus Map
3. Find Shortest Route (BFS)
4. Explore Campus (DFS)
5. Exit
==============================================
Enter your choice: 2
```

```
========== ANITS CAMPUS MAP ==========
Main Gate -> Open Auditorium, Canteen, Admin Block
Open Auditorium -> Main Gate
Canteen -> Main Gate
Admin Block -> Main Gate, CSE, ECE, EEE, Chemical, Mechanical
CSE -> Admin Block, ECE, EEE, Chemical, Mechanical
ECE -> Admin Block, CSE, EEE, Chemical, Mechanical
EEE -> Admin Block, CSE, ECE, Chemical, Mechanical
Chemical -> Admin Block, CSE, ECE, EEE, Mechanical
Mechanical -> Admin Block, CSE, ECE, EEE, Chemical, Food Engine, Bus Area
Food Engine -> Mechanical, Bus Area
Bus Area -> Mechanical, Food Engine, Hostels, Back Gate
Hostels -> Bus Area
Back Gate -> Bus Area


==============================================
```

```
 ==============================================
        ANITS CAMPUS ROUTE NAVIGATION
 ==============================================
 1. Display Locations
 2. Display Campus Map
 3. Find Shortest Route (BFS)
 4. Explore Campus (DFS)
 5. Exit
 ==============================================
 Enter your choice: 3

 ========== ANITS LOCATIONS ==========
 1. Main Gate
 2. Open Auditorium
 3. Canteen
 4. Admin Block
 5. CSE
 6. ECE
 7. EEE
 8. Chemical
 9. Mechanical
 10. Food Engine
```

```
11. Bus Area
12. Hostels
13. Back Gate
====================================

Enter starting location: Admin Block
Enter destination: Bus Area

========== SHORTEST ROUTE ==========

Admin Block -> Mechanical -> Bus Area

Number of roads: 2
==================================


================================================
       ANITS CAMPUS ROUTE NAVIGATION
================================================
1. Display Locations
2. Display Campus Map
3. Find Shortest Route (BFS)
4. Explore Campus (DFS)
5. Exit
================================================
Enter your choice: 3

========== ANITS LOCATIONS ==========
1. Main Gate
2. Open Auditorium
3. Canteen
4. Admin Block
5. CSE
6. ECE
7. EEE
8. Chemical
9. Mechanical
```

```
10. Food Engine
11. Bus Area
12. Hostels
13. Back Gate
====================================

Enter starting location: Bus Area
Enter destination: Admin Block

========== SHORTEST ROUTE ==========

Bus Area -> Mechanical -> Admin Block

Number of roads: 2
====================================



================================================
       ANITS CAMPUS ROUTE NAVIGATION
================================================
1. Display Locations
2. Display Campus Map
3. Find Shortest Route (BFS)
4. Explore Campus (DFS)
5. Exit
================================================
Enter your choice: 4

========== ANITS LOCATIONS ==========
1. Main Gate
2. Open Auditorium
3. Canteen
4. Admin Block
5. CSE
6. ECE
7. EEE
8. Chemical
9. Mechanical
```

```
10. Food Engine
11. Bus Area
12. Hostels
13. Back Gate
====================================

Enter starting location for DFS: Hostels

========== DFS CAMPUS EXPLORATION ==========
Hostels Bus Area Mechanical Admin Block Main Gate Open Auditorium Canteen CSE ECE
EEE Chemical Food Engine Back Gate
```

## 13. RESULTS AND ANALYSIS

The Campus Route Navigation system was successfully implemented using graph traversal techniques. The campus locations were represented as vertices and the roads connecting them were represented as edges.

The system successfully performs the following operations:

- Displays all available campus locations.
- Displays the campus map and connections.
- Finds the shortest route between two locations (in number of roads) using BFS.
- Performs campus exploration using DFS.
- Displays the number of roads in the shortest route.
- Shows an error message when an invalid location or an invalid menu choice is entered.

**Result**

For example, when the user selects Admin Block as the starting location and Bus Area as the destination, the BFS algorithm displays the route:

```
Admin Block -> Mechanical -> Bus Area
Number of roads: 2
```

The reverse request, Bus Area to Admin Block, gives Bus Area -> Mechanical -> Admin Block with 2 roads. Both outputs are shown in the screenshots in Section 12. As another example, when Main Gate is the starting location and Food Engine is the destination, BFS finds:

```
Main Gate -> Admin Block -> Mechanical -> Food Engine Number
of roads: 3
```

The DFS algorithm explores the connected campus locations starting from a selected location. From Hostels it visits all 13 locations in the order shown in Section 12.

**Verified Test Cases**

| No. | Menu option and input | Output |
|---|---|---|
| 1 | 3: Admin Block → Bus Area | Admin Block -> Mechanical -> Bus Area; Number of roads: 2 |
| 2 | 3: Bus Area → Admin Block | Bus Area -> Mechanical -> Admin Block; Number of roads: 2 |
| 3 | 3: Main Gate → Food Engine | Main Gate -> Admin Block -> Mechanical -> Food Engine; Number of roads: 3 |
| 4 | 3: Hostels → Back Gate | Hostels -> Bus Area -> Back Gate; Number of roads: 2 |
| 5 | 3: Main Gate → Foo (not a location) | Invalid location name! |
| 6 | 4: DFS from Hostels | Hostels Bus Area Mechanical Admin Block Main Gate Open Auditorium Canteen CSE ECE EEE Chemical Food Engine Back Gate |

| | | |
|---|---|---|
| 7 | 4: DFS from Main Gate | Main Gate Open Auditorium Canteen Admin Block CSE ECE EEE Chemical Mechanical Food Engine Bus Area Hostels Back Gate |
| 8 | Menu choice 9 | Invalid choice! Please try again. |

**Analysis:**

The results show that graph traversal is effective for representing and navigating campus routes. BFS is useful for finding the shortest route in terms of the number of roads when the graph is unweighted; it does not measure the physical distance between locations. DFS is useful for exploring the different connected locations in the campus; its output order depends on the order in which neighbours are stored.

Since the campus has only 13 locations and 23 roads, both traversals run instantly. With adjacency lists each traversal takes O(V + E) time, while the lookup of a location by name uses a linear search, which takes O(V) time and is acceptable for the small, fixed number of locations.

Therefore, the developed system provides a simple and efficient method for campus route navigation using graph traversal algorithms.

# 14. APPLICATIONS

The Campus Route Navigation system using graph traversal can be used in various situations where navigation between different locations is required.

### 14.1 Campus Navigation

It helps students, staff, and visitors find routes between important locations such as the main gate, department blocks, canteen, hostels, and the administrative block.

### 14.2 Finding Shortest Routes

BFS can be used to find a route with the minimum number of roads between two campus locations in an unweighted graph.

### 14.3 Visitor Navigation

The system can help new visitors easily identify the route from the Main Gate to important places such as the Admin Block or the Open Auditorium.

### 14.4 Emergency Navigation

Graph-based navigation can help identify possible routes between locations during emergency situations. (The present program does not model blocked roads; this would need an extension.)

### 14.5 Campus Planning

The graph representation can help understand the connections between different campus buildings and pathways.

### 14.6 Educational Purpose

The project can be used to demonstrate practical applications of graph data structures and traversal algorithms such as BFS and DFS.

### 14.7 Route Exploration

DFS can be used to explore connected campus locations and understand which locations can be reached from a given starting point.

Overall, the system demonstrates how graph traversal algorithms can be applied to solve real-world navigation problems within a campus.

## 15. ADVANTAGES

1. **Simple Navigation** – Helps users find routes between different campus locations easily.
2. **Shortest Route** – BFS can find the shortest route in terms of the number of roads in an unweighted graph.
3. **Easy to Understand** – The system uses simple graph concepts such as vertices and edges.
4. **Fast Traversal** – BFS and DFS traverse the campus graph in O(V + E) time.
5. **Easy to Implement** – The system is implemented using basic C programming, structures, arrays and recursion.
6. **Useful for Students and Visitors** – It helps new students, staff, and visitors understand campus routes.
7. **Educational Application** – Demonstrates the practical use of graph traversal algorithms.
8. **Modular Design** – Each task (adding locations, adding roads, displaying, BFS, DFS) is a separate function, which makes the program easy to read and extend.

## 16. LIMITATIONS

1. **No Real-Time Location** – The system does not track the user's current GPS location.
2. **Fixed Campus Map** – The locations and roads are predefined in the program; the menu cannot add or change them while the program runs.
3. **No Distance Calculation** – BFS considers the number of roads, not the actual distance of each road.
4. **No Traffic Information** – The system does not consider traffic, blocked roads, or crowded paths.
5. **Manual Updates** – Any changes to campus roads or locations must be updated in the program.
6. **Limited Locations** – The program supports only a fixed maximum number of locations (MAX = 20), and at most 20 neighbours for each location.
7. **No Graphical Map** – The current system displays routes as text rather than on a visual campus map.
8. **Exact Name Entry** – Locations must be typed with their exact, case-sensitive names; they cannot be selected by number.
9. **Limited Input Validation** – Non-numeric menu input and very long names are not handled (Section 11.1).
10. **Single Route** – Only one shortest route is displayed even if several routes have the same number of roads, and DFS only prints a visiting order.

## 17. CONCLUSION

The Campus Route Navigation Using Graph Traversal project successfully demonstrates how graph data structures and traversal algorithms can be applied to solve a real-world navigation problem. In this project, the 13 important locations of the ANITS campus are represented as

vertices, while the 23 roads or paths connecting them are represented as edges of an undirected, unweighted graph.

The graph is stored in a fixed-size array of Location structures, each containing the name of the location and an array-based adjacency list. The system uses Breadth First Search (BFS), implemented with an array-based queue, a visited array and a parent array, to find the shortest route between two campus locations in terms of the number of roads. Depth First Search (DFS), implemented recursively, is used to explore and traverse the connected locations of the campus. A menu-driven interface lets the user display the campus locations, view the campus map, find routes, explore the campus and exit.

The project shows that graph traversal provides a simple and effective method for representing and navigating campus routes. It also helps in understanding the practical implementation of important data-structure concepts such as graphs, structures, arrays, adjacency lists, queues, recursion, BFS, and DFS, with a running time of $O(V + E)$ for each traversal.

Although the current system has limitations such as fixed campus locations, no real-time GPS, no traffic or distance information, a text-only interface and limited input validation, it provides a strong basic model for campus navigation. In the future, the system can be enhanced by adding GPS-based location tracking, weighted graphs, actual distance calculation, real-time traffic information, interactive maps, and dynamic route updates.